SQL - Basiskurs

Dipl.-Ing. Leonard Guelmino

Trainer

Dipl.-Ing. Leonard Guelmino

Trainer

E leonard@alpinecoders.dev

Über mich

Informatik @TU Wien

Trainer @WIFI Wien & @mit.at

CTO in einem Startup

Lieblingssprachen: Python, Typescript, Kotlin, Rust

Ehrenamt: Rotes Kreuz Korneuburg

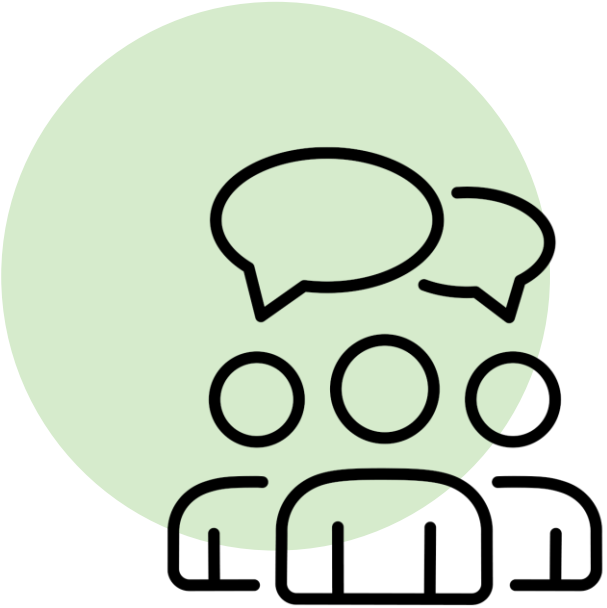
Hobby: Vogelkunde

Hallihallo!

Whoami?

Welches Vorwissen bringe ich mit?

Was erwarte ich mir von diesem Kurs?



Methodik

20/70/10

Theorie

Übung

Review

Vortrag

Alle Lerninhalte werden zuerst theoretisch behandelt.

Aufgaben

Nach (fast) jedem Theoriemodul folgt eine Übungseinheit, in dem die Theoire praktisch angewandt wird.

Musterlösung

Jede Übung wird anhand der Musterlösung gemeinsam durchbesprochen und Fragen beantwortet.

Curriculum

Montag

VM

Modul 1: Einführung & Architektur

Modul 2: ERM & Datenbankdesign

NM

Modul 3: DDL – Datenbanken definieren

Modul 4: Erste Selektionen & Syntax-Regeln

Dienstag

VM

Modul 5: Daten-Setup & Filterung

Modul 6: Joins – Tabellen vereinigen

NM

Modul 7: Aggregation & Gruppierung

Modul 8: Sortierung & Formatierung

Mittwoch

VM

Modul 9: DML – Daten verwalten

Modul 10: Mengenoperationen & Subqueries

NM

Modul 11: Fortgeschrittene Konzepte

Kursplattform kennenlernen und technisches Setup überprüfen

Übung 0
Setup

Modul 1

Einführung & Architektur

AKH Wien

© C.Stadler/Bwag; CC-BY-SA-4.0

- 1.706 Betten
- 8.960 Angestellte (davon 1.631 Ärzt:innen)
- Behandlungen pro Jahr (2021):

  61.016 Stationär

  515.687 Ambulant
- Quelle: Wikipedia

Vom Papierarchiv zur relationalen Datenbank

Lfd-Nr	Pat_Vorname	Pat_Nachname	SVNummer	Arzt_Name	Arzt_Fachrichtung	Station_Name	Diagnose	Kosten
1	Max	Mustermann	1234010180	Dr. Gruber	Kardiologie	Herzstation	Hypertonie	150
2	Erika	Schmidt	5678020285	Dr. Gruber	Kardiologie	Herzstation	Check-up	80
3	Max	Mustermann	1234010180	Dr. Wagner	Neurologie	Neurologie West	MigrÃ¤ne	120
4	Max	Mustermann	1234010180	Dr. Gruber	Kardio-Logie	Herzstation	Kontrolle	90

Vom Papierarchiv zur relationalen Datenbank

- Datenmanagement im klinischen Umfeld erfordert **Konsistenz** und **Redundanzfreiheit**.
- Excel-Listen skalieren nicht für 1.500 Ärzte und Pflegekräfte im parallelen Schreibzugriff.
- Ein **Relationales Datenbankmanagementsystem (RDBMS)** erzwingt **Struktur** und garantiert **Datenintegrität**.
- SQL: **Structured Query Language**

Architektur: Motor vs. Lenkrad

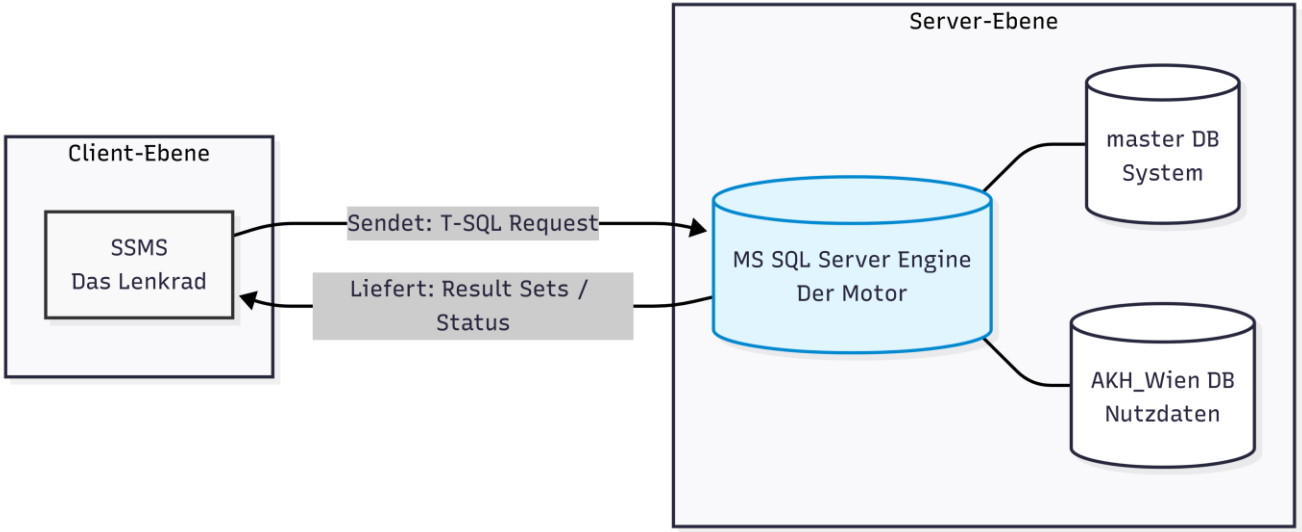
MS SQL Server & Management Studio (SSMS)

MS SQL Server (Der Motor):
Der Hintergrunddienst (sqlservr.exe). Er verarbeitet Daten, steuert Festplattenzugriffe und sichert Transaktionen.

SSMS (Das Lenkrad):
Die grafische Client-Anwendung. Sie sendet T-SQL-Befehle an den Server und visualisiert Ergebnisse.

Architektur: Motor vs. Lenkrad

MS SQL Server und Management Studio (SSMS)



Das Relationenmodell

Entitäten in Beziehung setzen

- Daten werden in strikt typisierten, zweidimensionalen Tabellen (**Entitäten**) strukturiert.
- Verknüpfungen zwischen Tabellen erfolgen über **logische Beziehungen**.

Das Relationenmodell

Entitäten in Beziehung setzen

- Die **Patienten**-Tabelle und die **Diagnosen**-Tabelle sind getrennt. Die Sozialversicherungsnummer (Primary Key) fungiert als universeller **Schlüssel** zur Verknüpfung.

Pat_ID	Vorname	Nachname	SVNummer
1	Max	Mustermann	1234010180
2	Erika	Schmidt	**5678020285**
3	Max	Mustermann	1234010180
4	Max	Mustermann	1234010180

Diagnose_ID	Arzt_Name	Arzt_Fachrichtung	Station_Name	Diagnose	SVNummer
1	Dr. Gruber	Kardiologie	Herzstation	Hypertonie	1234010180
2	Dr. Gruber	Kardiologie	Herzstation	sVT	**5678020285**
3	Dr. Wagner	Neurologie	Neurologie West	MigrÃ¤ne	1234010180
4	Dr. Gruber	Kardio-Logie	Herzstation	STEMI	1234010180

Datentypen

Text & Ganzzahlen

- **INT (Integer):** 4-Byte Ganzzahl. Hochperformant für System-IDs oder Stationsnummern.
- **CHAR(n):** Zeichenkette mit fixer Länge. Speichert Text effizient, da nur der tatsächlich benötigte Platz belegt wird (z.B. für SVNummer).
- **VARCHAR(n):** Zeichenkette variabler Länge (z.B. für Patientennamen).

Pat_ID	Vorname	Nachname	SVNummer
1	Max	Mustermann	1234010180
2	Erika	Schmidt	**5678020285**
3	Max	Mustermann	1234010180
4	Max	Mustermann	1234010180

Datentypen

Fließkommazahlen & Zeit

- **DECIMAL(p,s):** Fixkommazahl. Zwingend erforderlich für exakte Werte ohne Rundungsfehler. **DECIMAL(3,1)** speichert eine Körpertemperatur von 38.5 präzise.
- **DATETIME2 / DATE:** Chronologische Daten. Z.B. für Triage-Zeiten, Aufnahme, Entlassung und Medikamenten-Zyklen.

System-Initialisierung

Der erste Transact-SQL Befehl

Mit Data Definition Language (DDL) definieren wir die Systemstrukturen.

Wir trennen den logischen Arbeitsbereich explizit vom System-Kontext (master).

```sql
-- Erstellung der leeren Datenbank
CREATE DATABASE AKH_Wien;
GO

-- Kontextwechsel (verhindert Anlage in Systemdatenbanken)
USE AKH_Wien;
GO

-- Verifikation der Existenz (Aliasing ist Pflicht)
SELECT
    name AS DatabaseName,
    state_desc AS OperationalState
FROM sys.databases
WHERE name = 'AKH_Wien';
```

Modul 1 Stolperfallen

Best Practices

- Das Schließen des **SSMS** beendet den SQL Server *nicht*.
- Standardmäßig verbindet sich das SSMS mit der **master**-Datenbank. Ohne das Kommando **USE AKH_Wien** erstellen Sie Patienten-Tabellen versehentlich im kritischen Systembereich des Servers.
- Ein falsch gewählter Datentyp lässt sich im leeren System leicht ändern (**ALTER TABLE**). Bei 10 Millionen Patienten-Datensätzen führt dieselbe Änderung zu massiven Sperren (Table Locks) und Systemstillstand.

Modul 2

ERM & Datenbankdesign

Entitäten und Attribute

Das S in SQL

- **Entitäten** repräsentieren reale, abgrenzbare Informationsobjekte (z.B. Patient, Station).

- **Attribute** definieren die spezifischen, atomaren Eigenschaften vin Entitäten (z.B. Aufnahmedatum, Stationsname).

Primärschlüssel

Primary Key - PK

- Ein **PK** identifiziert jeden Datensatz einer Tabelle zweifelsfrei, eindeutig und unveränderlich.
- Auf Datenbankebene erzwingt der **PK** systemseitig die strikte Kombination der **UNIQUE** und **NOT NULL** Constraints.

Fremdschlüssel

Foreign Key - FK

- Ein **FK** in der Kind-Tabelle verweist zwingend auf den exakten **PK** der Eltern-Tabelle.
- Er garantiert **referentielle Integrität**. RDBMS blockieren Löschvorgänge, die Verweise ins Leere (Orphan Records) erzeugen würden.

Kardinalitäten

1:n Beziehung

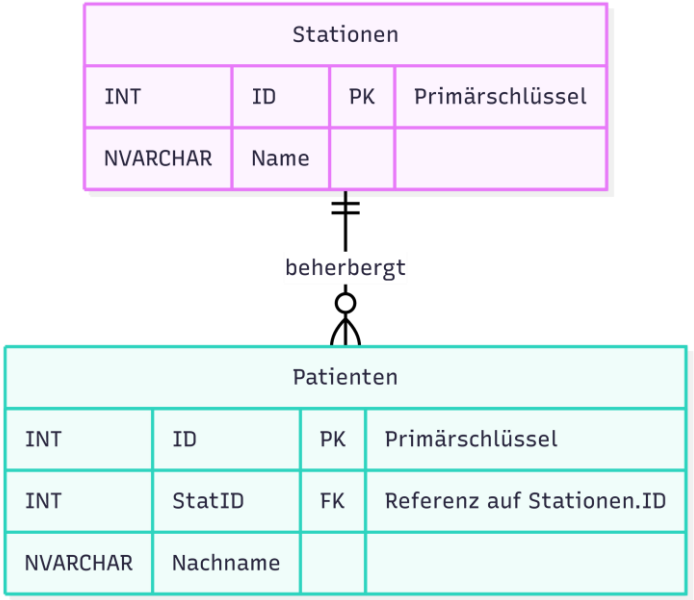
- **Kardinalitäten** definieren das quantitative Verhältnis zwischen zwei **Entitäten**.

- **1:n (One-to-Many):** Eine Station beherbergt zeitgleich **viele** Patienten, aber ein Patient liegt auf **exakt einer** Station.

Kardinalitäten

1:n Beziehung

- Der **Fremdschlüssel** liegt im RDBMS zwingend immer auf der "**n**"-Seite.



Kardinalitäten

m:n Beziehung

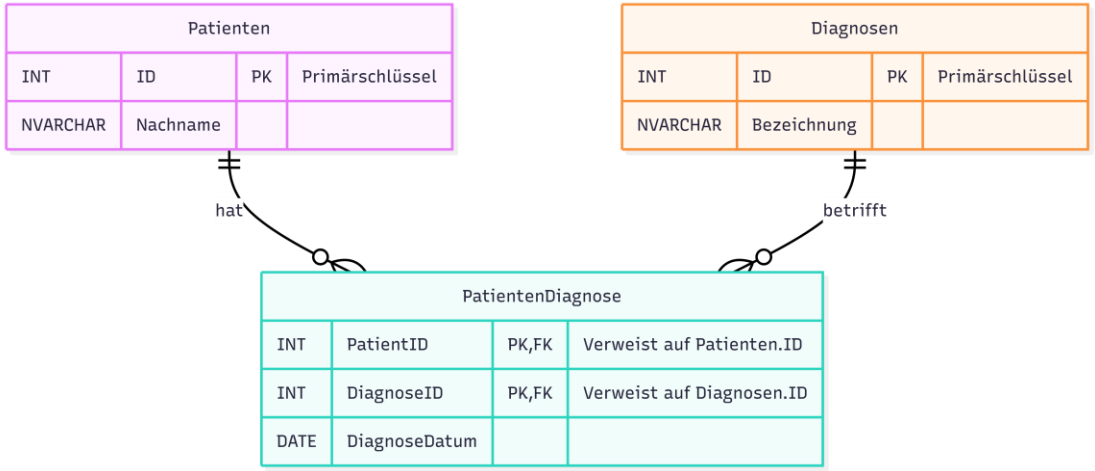
- **m:n (Many-to-Many):** Ein Patient hat im Lebenszyklus mehrere Diagnosen; eine spezifische Diagnose betrifft viele Patienten.

- **RDBMS können m:n-Beziehungen physisch nicht abbilden**. Sie erfordern eine zwingende Auflösung durch eine Mapping-Tabelle.

Kardinalitäten

m:n Beziehung

- Die Tabelle **PatientenDiagnose** speichert die Kombination aus **PatientID** und **DiagnoseID** und wandelt die Struktur in **zwei 1:n** Beziehungen um.



Kardinalitäten

1:1 Beziehung

- **1:1 (One-to-One):**.Ein Patient hat eine E-card. Eine E-card gehört einem Patienten.

- SVNR wird nicht in eine eigene Tabelle geschrieben

Praxis: SSMS Database Diagram Tool

Visuelle Modellierung vor der DDL-Phase

- Das Tool ermöglicht die grafische Konstruktion von **Tabellen**, **Datentypen** und **FK-Beziehungen** direkt in der Metadaten-Struktur der Datenbank.
- Validierung der relationalen Logik erfolgt visuell, bevor fehlerhafter DDL-Code manuell geschrieben wird.
- Änderungen im Diagramm generieren im Hintergrund automatisiert das korrekte **T-SQL-Skript** für die Schema-Anpassung.

Modul 2 Stolperfallen

Best Practices

⚠️ Tipp: Natürliche vs. Künstliche Schlüssel

- Verwenden Sie für **Primärschlüssel** in der Praxis immer künstliche Schlüssel wie **INT IDENTITY(1,1)**.
- **Natürliche Schlüssel** scheitern an der Realität: Ein bewusstloser Notfallpatient (John Doe) hat initial keine SVN. Da ein **PK** aber zwingend **NOT NULL** erfordert, würde das System die Aufnahme des Datensatzes technisch verweigern.

ERM im SSMS

Übung 1
ERM & Datenbankdesign

Modul 3

DDL

Die Geburtsstunde der Datenbank

Vom Modell zur physischen Struktur

- **Data Definition Language (DDL)** übersetzt das visuelle **ER-Modell** in persistente Systemstrukturen.

- **CREATE TABLE** reserviert Speicherplatz und erzwingt das exakte Metadaten-Schema im SQL Server.

CREATE TABLE: Stammdaten Station

Aufbau der referenzierten Entität (Eltern-Tabelle)

- Relationales Design diktiert die Erstellungsreihenfolge: Entitäten ohne Fremdschlüssel müssen zwingend zuerst instanziiert werden.

- Jeder Spalte wird ein strikter Datentyp und ein Nullability-Zustand zugewiesen.

```sql
CREATE TABLE Station (
    StatID INT IDENTITY(1,1) PRIMARY KEY,
    StationName VARCHAR(50) NOT NULL,
    Stockwerk TINYINT NOT NULL
);
```

CREATE TABLE: Entität Patient

Fremdschlüssel und fixe Längen (Kind-Tabelle)

Die Definition der Kind-Tabelle inkludiert die harte, physische FK-Beziehung zur Tabelle Station.

Der Datentyp CHAR(10) garantiert die exakte, unveränderliche Länge der österreichischen Sozialversicherungsnummer (SVN).

Dynamische Längen (VARCHAR) sind hier technisch ineffizient.

```sql
CREATE TABLE Patient (
    PatID INT IDENTITY(1,1) PRIMARY KEY,
    Nachname VARCHAR(50) NOT NULL,
    Vorname VARCHAR(50) NOT NULL,
    GebDatum DATE NOT NULL,
    SVNummer CHAR(10) UNIQUE,
    StatID INT FOREIGN KEY REFERENCES Station(StatID)
);
```

ALTER & DROP: Laufende Anpassungen

Operationen am offenen System

ALTER TABLE mutiert bestehende Tabellenstrukturen (Spalten-Addition, Datentyp-Wechsel) ohne Datenverlust im laufenden Betrieb.

DROP TABLE vernichtet die Tabellenstruktur mitsamt aller darin enthaltenen Daten irreversibel auf Festplattenebene.

```sql
-- Erweiterung der Patientenakte um eine optionale Kontaktmöglichkeit
ALTER TABLE Patient
ADD Telefonnummer VARCHAR(20) NULL;

-- Irreversible Löschung (Vorsicht!)
-- DROP TABLE Station;
```

Constraints

Regelwerke auf Datenbankebene

- **DEFAULT**: Vergabe eines Standardwertes.

```sql
CREATE TABLE Behandlungen (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    -- ....
    Status NVARCHAR(50) DEFAULT 'Abgeschlossen' -- <--- DEFAULT Constraint
);
```

Constraints

Regelwerke auf Datenbankebene

- **NOT NULL**: Verhindert unvollständige Datensätze (z.B. ein Patient *muss* zwingend ein Geburtsdatum haben).

```
ALTER TABLE Patienten
ALTER COLUMN GebDatum DATE NOT NULL;
```

Constraints

Regelwerke auf Datenbankebene

- **UNIQUE**: Garantiert systemweite Eindeutigkeit für Nicht-PK-Attribute. Die SVN darf nur exakt einmal existieren.

```
ALTER TABLE Patienten
ADD CONSTRAINT UQ_Patienten_SVNummer UNIQUE (SVNummer);
```

Constraints

Regelwerke auf Datenbankebene

- **CHECK**: Validiert Wertebereiche direkt beim Schreibvorgang um Tippfehler zu verhindern.

```
ALTER TABLE Stationen
ADD CONSTRAINT CHK_Stationen_Stockwerk CHECK (Stockwerk >= 0 AND Stockwerk <= 20);
```

Modul 3 Stolperfallen

Best Practices

⚠️ Achtung: Referentielle Integrität blockiert Zerstörung

- Führen Sie **DROP TABLE Station;** aus, wird der SQL Server dies mit einer Fehlermeldung quittieren, sofern Datensätze in der Tabelle Patient existieren.
- Das **RDBMS** schützt sich selbst: Eine **Station** kann nicht abgerissen werden, solange **Patienten** darauf **referenzieren**. Zuerst müssen die Patienten verlegt (**geändert**) oder entlassen (**gelöscht**) werden.

Skripting & Ausführung im SSMS

Übung 2
DDL

Modul 4

Erste Datenabfragen & Syntax-Regeln

Dateneingabe

INSERT INTO ‹Tabelle› (‹Attributname›) VALUES (‹Attributwert›)

INSERT INTO injiziert physische Datensätze in die zuvor definierte Tabellenstruktur.

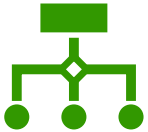
Das RDBMS validiert jeden Datensatz in Echtzeit strikt gegen die definierten Constraints (NOT NULL, UNIQUE, FOREIGN KEY).

Ein Verstoß (z.B. die Eingabe einer bereits existierenden Sozialversicherungsnummer) führt zu einem **Rollback** der gesamten **Transaktion**.

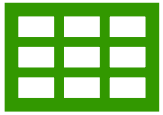
```
-- Anlage der Eltern-Entität (Station)
INSERT INTO Station (StationName, Stockwerk)
VALUES ('Kardiologie', 3);

-- Anlage der Kind-Entität mit Referenz (StatID = 1)
INSERT INTO Patient (Nachname, Vorname, GebDatum, SVNummer, StatID)
VALUES ('Müller', 'Anna', '1985-05-20', '1234200585', 1);
```

Datenabfrage

SELECT ‹Attribut› FROM ‹Tabelle›



SELECT definiert exakt, *welche* Spalten (Attribute) das Result Set (die Ergebnismenge) enthalten soll.



FROM spezifiziert die exakte Quelle (die physische Tabelle), aus der die Daten gelesen werden.

```
SELECT
    SVNummer AS Sozialversicherungsnummer,
    Nachname AS Familienname
FROM Patient;

```

Aliasing und Literale

SELECT ‘<Literal>‘ AS <Alias> FROM <Tabelle>

- **Spaltennamen** im System (z.B. GebDatum) sind aus Gründen der Namenskonvention für Endanwender oft kryptisch.
- Das Schlüsselwort **AS** erzeugt einen temporären Spalten-**Alias**, der ausschließlich für die finale Anzeige gerendert wird.
- **Literale** (fest codierte Text- oder Zahlenwerte) reichern das Result Set um statische Informationen an, die nicht in der Tabelle existieren.

```sql
SELECT
    'AKH Wien' AS Klinikstandort,
    Nachname AS PatientenNachname,
    Vorname AS PatientenVorname
FROM Patient;
```

Berechnungen zur Laufzeit

Dynamische Informationsgewinnung

- **T-SQL** kann mathematische Operationen **(+, -, *, /)** oder **Datumsfunktionen** direkt in der **SELECT**-Klausel ausführen.
- Berechnete Spalten existieren nur im flüchtigen RAM (**Result Set**) und belegen keinen Speicherplatz auf der Festplatte.

```sql
SELECT
    Nachname AS Familienname,
    YEAR(GETDATE()) - YEAR(GebDatum) AS GeschaetztesAlter
FROM Patient;
```

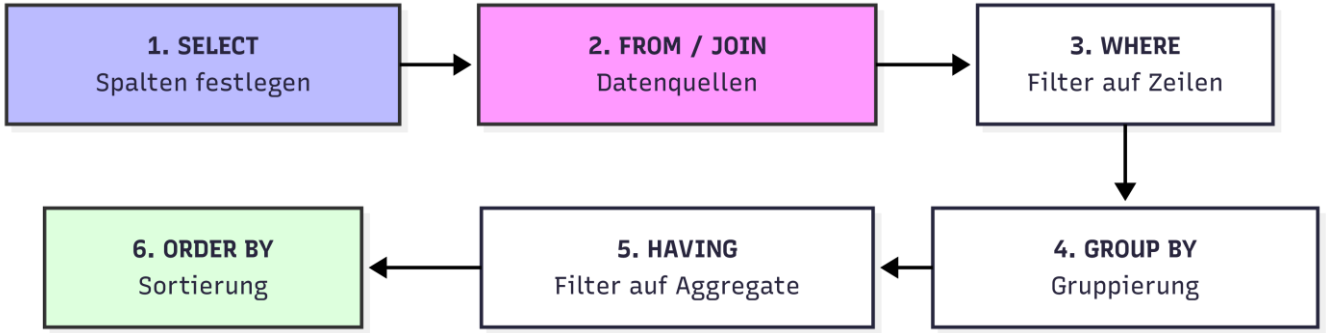
Die Engine denkt anders als wir schreiben

Lexikalische vs. Logische Verarbeitungsreihenfolge

- Code wird lexikalisch *geschrieben* (**SELECT -> FROM -> WHERE**), aber von der Engine logisch abweichend *ausgeführt*.
- Die Ausführung startet zwingend bei **FROM** (Woher kommen die Daten?), bevor **SELECT** (Was brauche ich davon?) greift.
- **Konsequenz:** Wenn wir in Filter setzen (WHERE), kennt der Server die in der SELECT-Klausel definierten Aliase technisch noch gar nicht. Ein Filter auf **WHERE GeschaetztesAlter > 50** wirft daher unweigerlich einen Syntaxfehler.

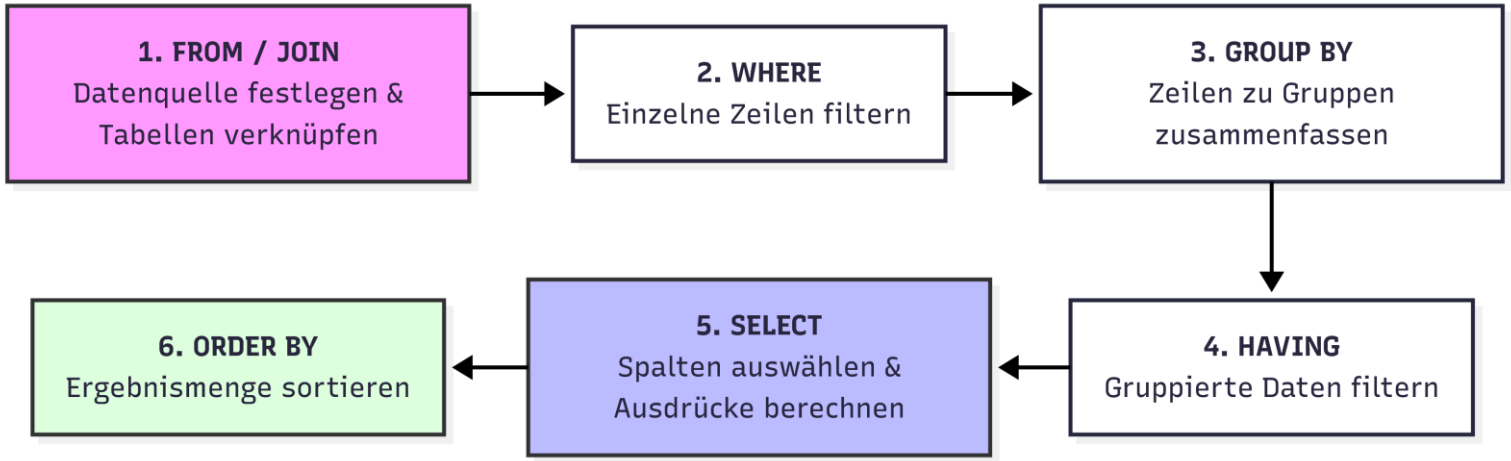
T-SQL

Lexikalische Reihenfolge



T-SQL

Processing Reihenfolge



Modul 4 Stolperfallen

Best Practices

⚠️ Achtung: Der SELECT * Anti-Pattern

- Ein **SELECT *** (Gib mir alle Spalten) ist in der Produktionsumgebung strengstens verboten.
- Es zerschießt die Applikationsobjekte (wenn nachträglich neue Spalten in der Tabelle hinzugefügt werden) und verschwendet massiv Netzwerkbandbreite, da Spalten übermittelt werden, die das Frontend womöglich gar nicht benötigt.
- Schreiben Sie Spalten in produktivem Code immer **explizit** aus.

Übung 3
SELECT & Syntax-Regeln